

Grand Finale
CIS 350: Advanced Data Structures and Algorithms
Write Up

For my final project I chose to implement the bidirectional search algorithm. This method works by expanding nodes from both directions at the same time. One starts at the beginning position and the other starts from the goal position. Each side uses a queue to perform a BFS traversal of neighboring nodes. When one search reaches a node that has already been visited by the opposite search, the algorithm sets that node as the meeting point. This point marks the location where a path is to be reconstructed by tracing parent nodes from the meeting point back to the start and goal.

The algorithm uses two queues, two visited node maps, and two parent tracking maps to allow both searches to operate independently while detecting when they intersect. The advantage to this algorithm is that the overall time complexity is significantly improved compared to regular BFS, which has a time complexity of $O(b^d)$. The bidirectional search reduces this to approximately $O(b^{(d/2)})$ where “b” is the branching factor and “d” is the depth of the solution. This change is due to the two searches only having to travel half the distance before meeting which means the number of explored nodes is cut down.

To implement the program, I used C++ with a grid-based environment which consisted of open spaces, walls or obstacles, a start node, and a goal node. I used queues from the standard template library (STL) to manage BFS traversal. I also used unordered maps to store visited

nodes and parent relationships for path reconstruction. The algorithm was able to check neighboring positions in four directions (up, down, left, and right). When a valid neighboring node was found, it was added to the queue and marked as visited. Once both searches reached the same node, the final path was reconstructed and displayed on the grid.

The bidirectional search algorithm has many advantages. For one, it is faster than regular BFS in most cases by exploring fewer nodes. It is capable of finding the shortest path in an unweighted graph and is very efficient in larger search spaces. The disadvantages would be that it is more complicated than the standard BFS. It also requires additional memory when using the two searches.

Bidirectional search offers many practical uses in real-world systems that require efficient pathfinding and navigation. Similar concepts are used in GPS routing systems, robotics, video game AI, and network communication where finding an efficient route between two points is necessary. The implementation of this project was to simplify the basic use of the algorithm to focus on the core concept rather than large-scale optimization. While this version operates on a small grid using basic data structures, the same methods can be applied to much more advanced systems that might use weighted graphs, massive datasets, real-time obstacle detection, or heuristic optimizations.